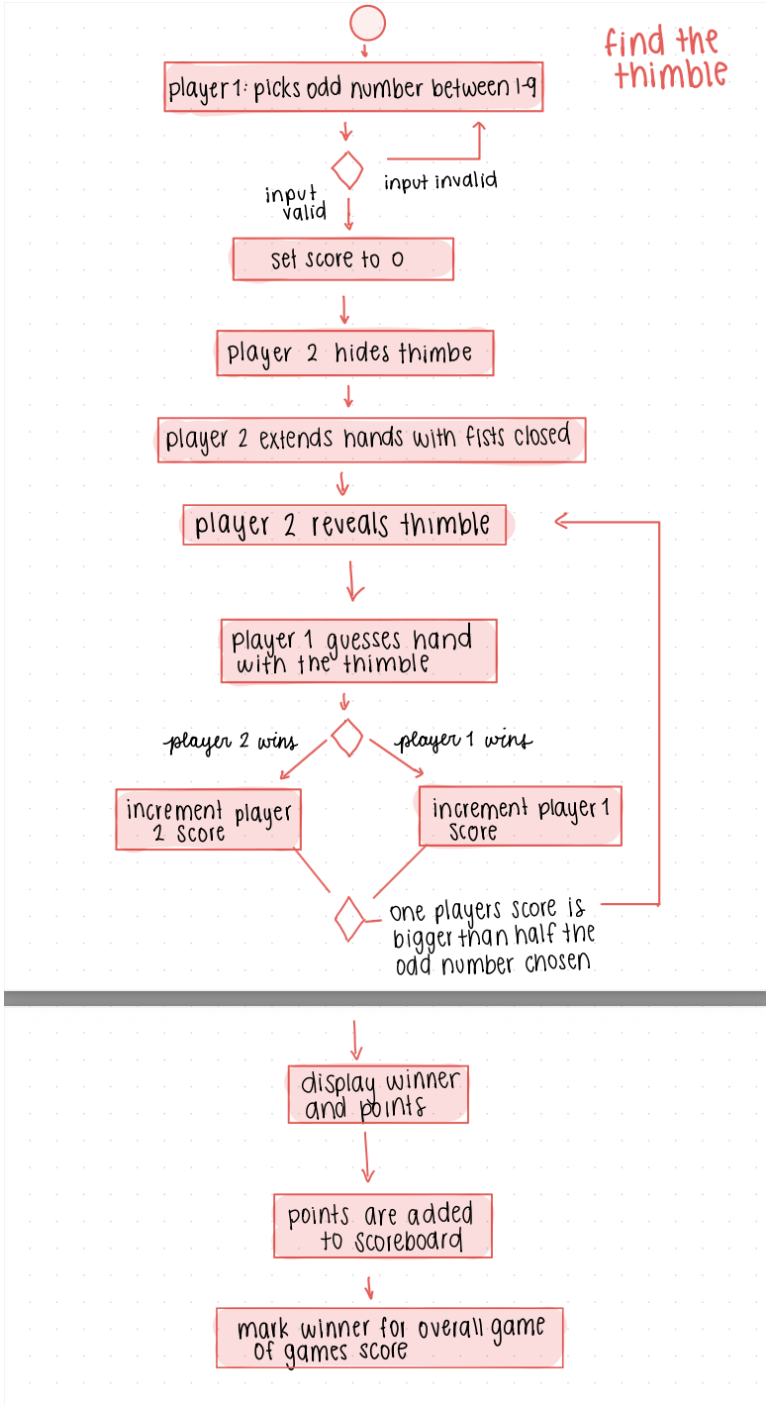
Activity DIagram - Find the thimble



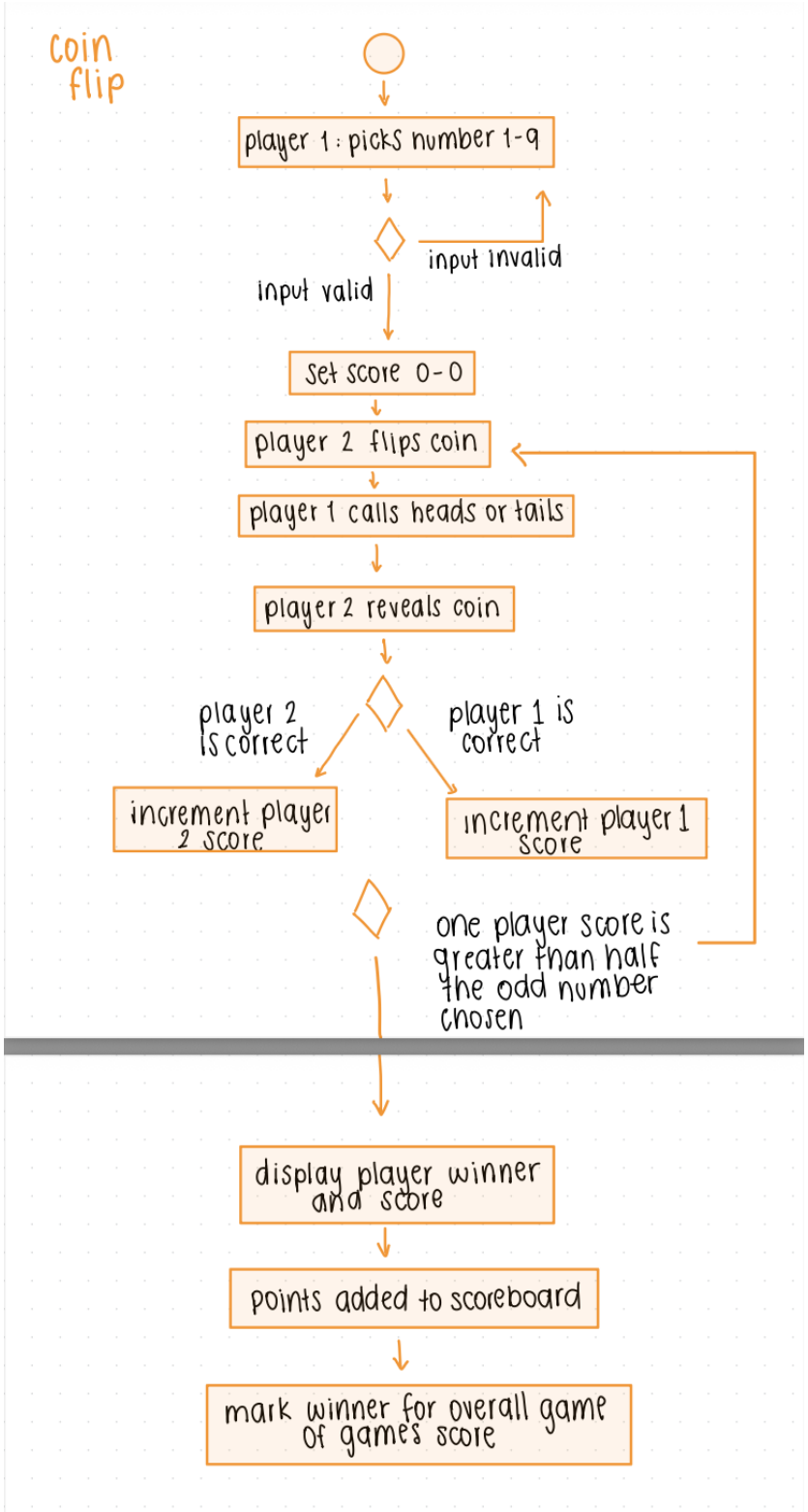
Use Case For Find The Thimble Game

(I'm making the game one class that can be at the menu for the overall game of games, I also just make it if statements but I can change it to classes for each small thing such as adjusting score or playing the game if needed)
highlighted text = stuff we can change later to match and be consistent

- For the Find the Thimble game, first make a class for the Find the Thimble game.

- Add 2 integer variables for player 1's score and the computer's score that are set to 0
- Print the menu "Welcome to the Find the Thimble Game, (explain the rules of the game to the user) for example:
you will best of up to 9 games where you have to guess if the thimble is in the left or right hand. If you guess correctly, you get one point, if you don't the computer wins"
- Prompt the user to pick an amount of rounds to play up to between 1 and 9, odd numbers only.
    - Repeatedly prompt the user if they pick an even number between 2 and 8 or if the number is not in between 1 and 9
    - Set that number to a variable for the number of rounds to play
- The computer makes a random number between 1 and 2 using the java.util.random where
    - 1 represents left and 2 represents left
    - Set that number to variable for the user to guess
- Prompt the user to guess between left or right by asking "Guess if the thimble is on the left or right ("L or "R")
    - Repeat until the user responds with either "L" or "R"
    - If the user guesses correctly
        - Increment the score of Player 1 by 1 point
        - check to see if player 1's score is bigger than half of the odd number chosen
            - If players points is bigger than half of the odd number chosen,
                - display that player 1 wins and both players points
                - Add the a point to overall points for the game of games point system for the user who won the Find the Thimble Game
            - If not, randomise the number for left and right for the computer and re-prompt the user to guess again
    - If the computer guesses correctly
        - Increment the score of the computer by 1 point
        - check to see if the computers score is bigger than half of the odd number chosen
            - If the computers score is bigger than half of the odd number chosen
                - display that the computer wins and both players points
                - Add a point to overall points for the game of games point system for the user who won the Find the Thimble Game
            - If not, randomise the number for left and right for the computer and re-prompt the user to guess again

Activity Diagram - Coin Flip



Use Case For Coin Flip

(same stuff as Find the thimble, I can adjust it with classes or other things later)

For the Coin Flip Game, first make a class for the Coin Flip Game.

- Add 2 integer variables for player 1's score and the computer's score that are set to 0
- Print the menu "Welcome to the Coin Flip Game (then explain rules)
- Prompt the user to pick an amount of rounds to play up to between 1 and 9, odd numbers only.
    - Repeatedly prompt the user if they pick an even number between 2 and 8 or if the number is not in between 1 and 9
    - Set that number to a variable for the number of rounds to play
- The computer makes a random number between 1 and 2 using the java.util.random where
    - 1 represents heads and 2 represents tails
    - Set that number to heads or tails depending on what it is, so that it can be displayed to the user
- Prompt the user to guess between heads or tails by asking "Call heads or tails ("H" or "T")
    - Repeat until the user responds with either "H" or "T"

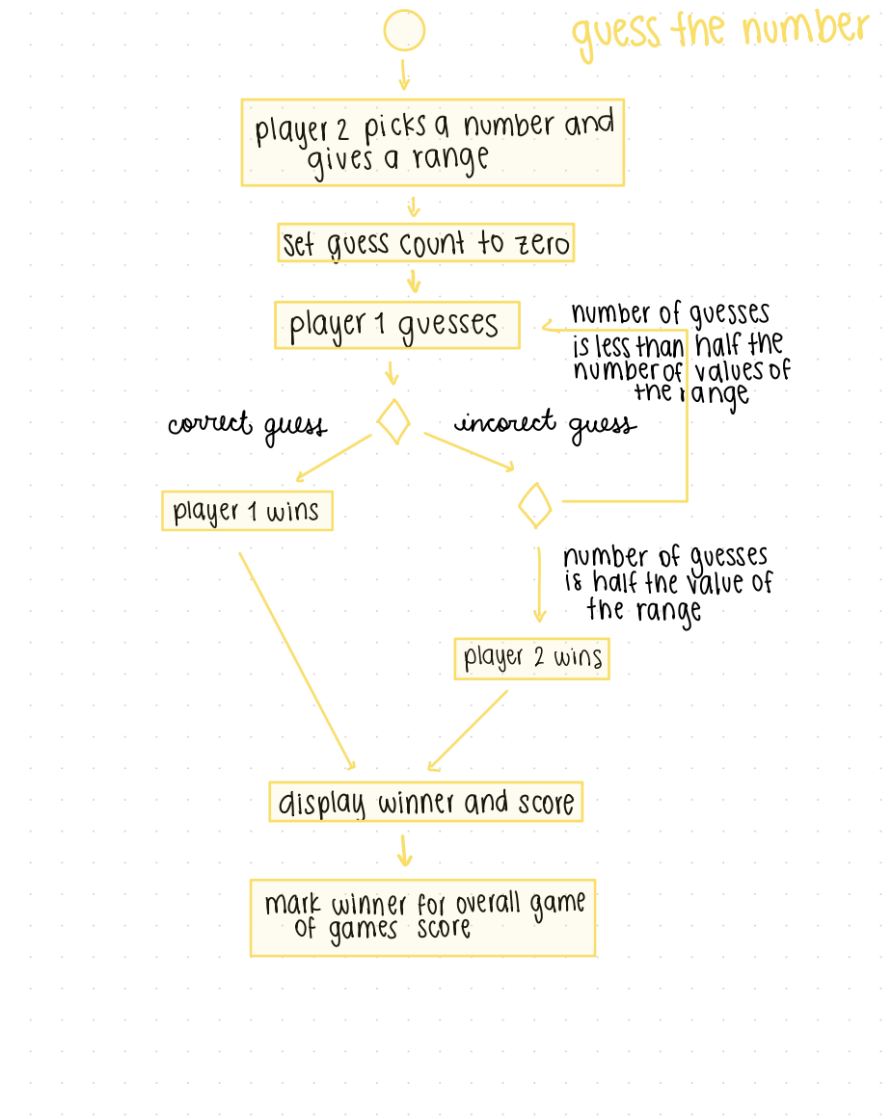
-If the user is correct

- Increment the score of Player 1 by 1 point
- check to see if player 1's score is bigger than half of the odd number chosen
    - If players points is bigger than half of the odd number chosen
        - display that player 1 wins and both players points
        - Add a point to overall points for the game of games point system for the user who won the Coin Flip Game
    - If not, randomise the number for left and right for the computer and re-prompt the user to guess again

- If the computer guesses correctly

- Increment the score of the computer by 1 point
- check to see if the computers score is bigger than half of the odd number chosen
    - If the computers score is bigger than half of the odd number chosen
        - display that the computer wins and both players points
        - Add a point to overall points for the game of games point system for the user who won the Coin Game
    - If not, randomise the number for left and right for the computer and re-prompt the user to guess again

Activity Diagram - Guess the Number

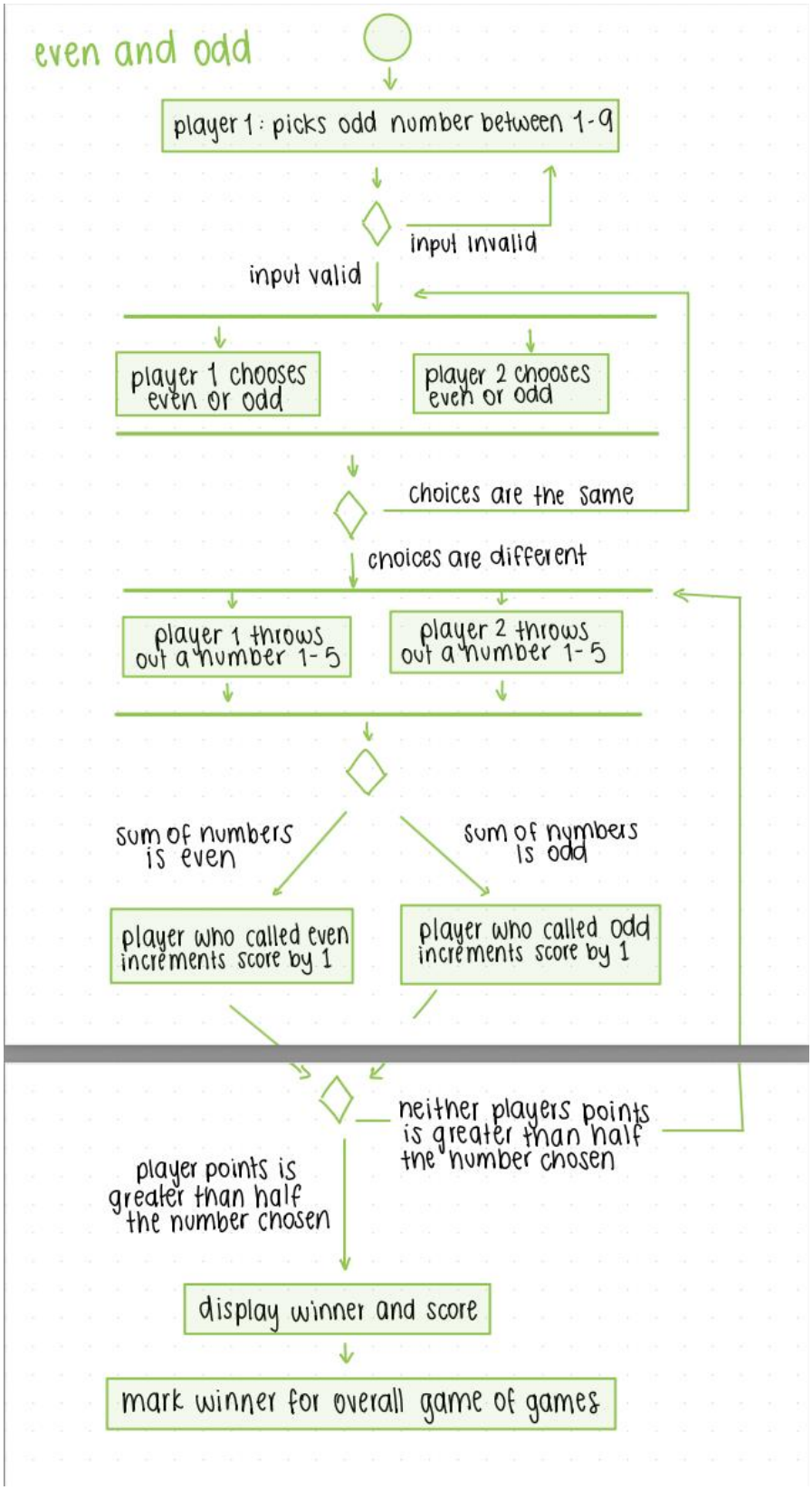


Use Case for Guess the Number

- Make class named GuessTheNumber
- Initialize variables:
    - lowestInRange: The lowest number in range.
    - highestInRange: The highest number in range
    - correctNumber: The number that the computer chooses at random for the user to guess.
    - guessLimit: The most amount of guesses the user has
    - guessCount: The number of guesses the user has so far
    - guessNumber: The number that the user guessed/chose in the given round
- Print the menu "Welcome to the Guess the Number Game (then explain rules)

- Computer will generate a range by randomly generating integers lowestInRange and highestInRange
- Set guessLimit to ((highestInRange - lowestInRange) / 2)
- Computer will generate an integer for correctNumber for user to guess
- Show the range and guess limit to the user.
- Prompt the user to give their guess.
    - If user input invalid
        - Repeatedly prompt user to guess until input is valid
    - If user input is valid
        - Increment guessCount by 1
        - Set guessNumber to user's input
- If guessNumber equals correctNumber (user's input/guess is correct)
    - Display user as winner
    - Increment user's score in overall game of games score by 1
    - Show option for user to play again or return to main menu
        - If user chooses to play again
            - Loop back to "Print the menu "Welcome to the Guess the Number Game (then explain rules)"
        - If user chooses main menu
            - Return to main menu
- If guessNumber does not equal correctNumber (user's input/guess is incorrect)
    - If guessCount equals guessLimit (user has reached their limit in guesses)
        - Display computer as winner
        - Increment computer's score in overall game of games score by 1
        - Show option for user to play again or return to main menu
            - If user chooses to play again
                - Loop back to "Print the menu "Welcome to the Guess the Number Game (then explain rules)"
            - If user chooses main menu
                - Return to main menu
    - If guessCount does not equal guessLimit (user has not reached their limit in guesses)
        - Loop back to "Prompt the user to give their guess" and repeat the following steps

Activity Diagram - Even and Odd

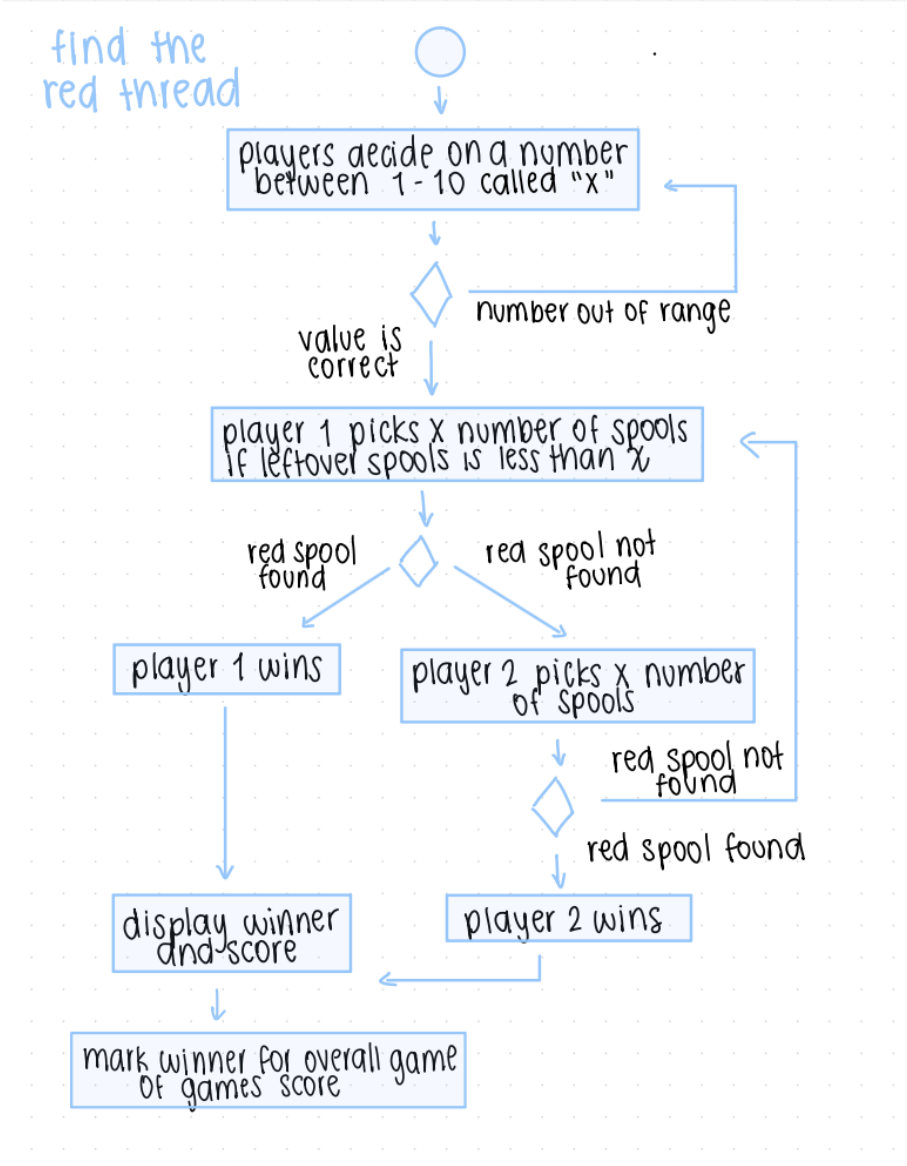


Use Case for Even Odd Game

- Make class named EvenOdd
- Initialize variables:
    - rounds: The number of rounds to be played
    - userScore: Tracks current score of user, set to 0.
    - compScore: Tracks current score of computer, set to 0.
    - userNumber: Number that user chose in given round
    - compNumber: Number that computer generates in given round
    - userGuess: A string that sets user's choice of even or odd
- Print the menu "Welcome to the Guess the Number Game (then explain rules)
- Prompt the user to pick an amount of rounds to play up to between 1 and 9, odd numbers only.
    - Repeatedly prompt the user if they pick an even number between 2 and 8 or if the number is not in between 1 and 9
    - Set user's input to variable "rounds"
- Prompt the user to guess between even or odd by asking "Choose even or odd ("E" or "O")
    - If input is invalid
        - Repeatedly prompt user for input until they respond with either "E" or "O"
    - If input is valid
        - Set userGuess to user's input
        - If user's input is "E", computer will show message: "You chose even, so I will choose odd"
        - If user's input is "O", computer will show message: "You chose odd, so I will choose even"
- Computer will generate random number from 1-5
    - Set compNumber to generated number
- Prompt user to enter a number between 1-5
    - If input is invalid
        - Repeatedly prompt user until input is between 1-5
    - If input valid
        - Set userNumber to user's input
- Set variable "sum" to the sum of compNumber and userNumber
    - If sum is even and userGuess is "E"
        - Increment userScore by 1
    - If sum is odd and userGuess is "O"
        - Increment userScore by 1
    - If neither of the above is true
        - Increment compScore by 1
- If userScore is greater than (rounds / 2) (user won the most rounds)
    - Display user as winner
    - Increment user's score in overall game of games score by 1
    - Show option for user to play again or return to main menu
        - If user chooses to play again
            - Set all integer variables to 0

            - Loop back to “Print the menu “Welcome to the Even Odd Game (then explain rules)”
        - If user chooses main menu
            - Return to main menu
- If computerScore is greater than (rounds / 2) (computer won the most rounds
    - Display computer as winner
    - Increment computer’s score in overall game of games score by 1
    - Show option for user to play again or return to main menu
        - If user chooses to play again
            - Set all integer variables to 0
            - Loop back to “Print the menu “Welcome to the Even Odd Game (then explain rules)”
        - If user chooses main menu
            - Return to main menu
- If neither userScore or computerScore is greater than (rounds / 2) (neither player won the most rounds)
    - Loop back to “Computer will generate random number from 1-5” and repeat the following steps

Activity Diagram - Find the Red Thread



Use Case for Find the Red Thread

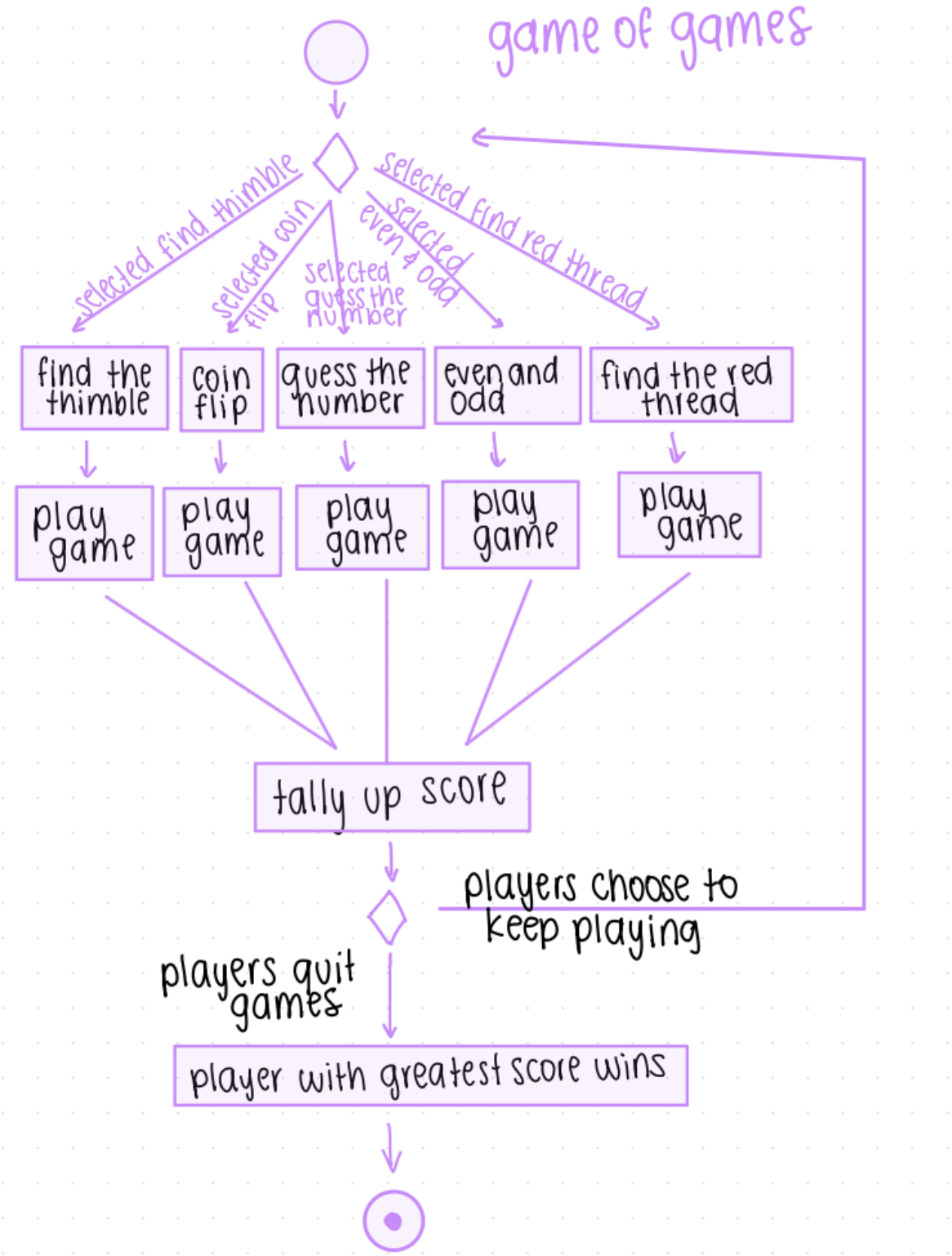
- Make class named FindTheRedThread
- Initialize variables:
- **playerScore**: The player's score at the beginning of the game. Initialized at 0 and increases when the player wins
- **computerScore**: The computer's score at the beginning of the game. Initialized at 0 and increases when the computer wins
- **totalSpools**: Total number of spools in the game. Initialized at 50
- **remainingSpools**: Number of spools left in the game. Initialized at 50 and decreases as spools are picked

- **xChoice**: The number of spools picked per turn (chosen by the user between 1 and 10)
- **redSpoolPosition**: The position of the red spool randomly generated between 1 and 50
- **gameOver**: Boolean variable initialized as false. Becomes true when the red spool is found and ends the game
- **playAgainChoice**: Stores user input ("Y" or "N") to determine if the game restarts

- Print the menu “Welcome to the Find the Red Thread Game (then explain rules)”
- There are 50 spools. One of them is red.
  - You are playing against the computer.
  - You will choose a number **X** between 1 and 10.
  - Both you and the computer will pick **X** spools per turn.
  - If the red spool is picked, that player wins immediately.
- Prompt the user to choose a number between 1 and 10
- If user input is invalid:
  - Repeatedly prompt player until input is a valid number between 1 and 10
- If user input is valid:
  - Set xChoice to the players input
  - Set **remainingSpools = totalSpools**
  - Generate random number for **redSpoolPosition** between 1 and 50
  - Start game loop while **gameOver == false**
- Display remaining spools and prompt user to press **ENTER** to pick spools
  - Player picks **xChoice** spools
- If player finds **redSpoolPosition**:
  - Display user as the winner
  - Increment **playerScore** by 1
  - Increment user’s score in overall game of games score by 1
  - Set **gameOver = true**
  - Show option for user to play again or return to main menu
- If player does not find **redSpoolPosition**:
  - Reduce **remainingSpools** by **xChoice**
- If **gameOver == false:**
  - Simulate computer picking **xChoice** spools
- If computer finds **redSpoolPosition**:
  - Display computer as winner
  - Increment computerScore by 1
  - Increment computer’s score in overall game of games score by 1
  - Set **gameOver = true**
- Show option for user to play again or return to main menu
- If computer does not find **redSpoolPosition**:
  - Reduce **remainingSpools** by **xChoice**
- If **gameOver == false:**
  - Loop back to “Display remaining spools and prompt user to press ENTER to pick spools”
- If user chooses to play again after the round is over:

    - Reset **gameOver** to false
    - Reset **remainingSpools** to 50
    - Loop back to “Print the menu Welcome to the Find the Red Thread Game”
- If user chooses main menu:
    - Return to main menu

Activity Diagram - Game of Games



Use Case for Overall Game of Games

- Set class name to: GameOfGames

- Initialize variables:
    - userTotalWins: tracks the number of full games won by player, initially 0
    - compTotalWins: tracks the number of full games won by the computer, initially 0
    - gamesPlayed: tracks the total number of game sessions completed, initially 0
- Main Menu
    - Displays: “Welcome to the Game of Games! Select a game to play or quit to see the final score.”
    - The system provides the following choices based on the activity diagram:
        - Find the Thimble
        - Coin Flip
        - Guess the Number
        - Even and Odd
        - Find the Red Thread
        - Quit Games
- User Selection
    - The system prompts the user to enter a choice from 1-6
    - If input is invalid, the system repeatedly prompts the user until a valid number is entered
    - If input is 1-5, the system branches to the selected game logic
    - If input is 6, the system proceeds to final tally
- Play Selected Game
    - The system executes the specific “Play Game” sequence for the chosen activity
    - Each game runs its internal rounds until a winner for that specific game is determined
- Tally up Score and Display Scoreboard
    - Upon returning from a game, the system updates the counts
        - Increment gamesPlayed by 1
        - Increment userTotalWins if the user won the game
        - Increment compTotalWins if the computer won the game
    - The system displays the current scoreboard
        - “Games Played: [gamesPlayed]”
        - “Your Total Wins: [userTotalWins]”
        - “Computer Total Wins: [compTotalWins]”
    - Loop: the system returns to the Main Menu sequence to allow the players to keep playing
- Final Results
    - When the user chooses to “quit games,” the system displays the final scoreboard
    - Determine overall winner:
        - If userTotalWins > compTotalWins: Display “You won the Game of Games!”
        - Else if compTotalWins > userTotalWins: Display “The Computer won the Game of Games!”
        - Else: Display “The Game of Games ended in a tie!”

The program terminates

DROP AS BOTH A WORD AND A PDF

Meeting Notes, Project: Game of Games 1

Meeting Number:	**Meeting Date:**	**Start Time:**	**End Time:**
1	4/3/26	11:00am	1:00pm

Attendee Name (Indicate note taker with *)	**Timeliness (on time, late, left early, missing)**
Allan	On time
David	On time
Erica*	5 min late
Valeria	On time

Purpose: (briefly describe general purpose of meeting expected outcomes)
Meeting to review the game of games assignment and discuss what tasks to do together

Agenda: (briefly outline plan for the meeting, i.e. what actions will be taken, expected outcomes if appropriate)

- Review the games played, its rules and important notes to consider
- Draw activity diagrams on as a group
- Develop use cases for each game

Major Accomplishments: (briefly describe what was achieved the meeting)

- Created activity diagrams for every game

Next Steps: (identify the steps to be taken next, and identify who is responsible to carrying out those steps if appropriate)

- We will discuss the activity diagrams and use cases as a group together

Meeting Notes: (Notes are not a direct transcription, but a condensed, as I would say somewhat briefly but completely, record of what was said and happened at the meeting. A major purpose of project notes is to have an accurate record of information, discussion points and decision influences and decisions made that can be referenced later if needed)

- David and Allan drew on the whiteboard for the group to see the activity diagram
- Valeria redraws diagram on word document for easy shareability/submission
- The entire group discusses the exact tasks needed for each game
    - We decided to name the two entities/players as “Player1” and “Player2” for easy readability.
    - After each game ends, a winner is shown for the individual game and the overall score for games of games is also updated
    - Sometimes, we realized that we would have to set variables before the start of the game, such as scores, or add extra steps for clarity.

Documents: (included or links to them)